# SL Project

Chiara Mensini, Lorenzo Bonatti, Sara Cassiani

2024-03-16

## Contents

**1. Descriptive Analysis** **2**

1.1 The dataset . . . . . 2

1.2 Goal of the project . . . . . 3

**2. Data preparation** **3**

2.1 Check for NA . . . . . 3

2.2 Dataset reduction . . . . . 4

2.3 Categorical variables . . . . . 4

**3. Plots** **4**

**4. Logistic regression** **5**

4.1 Logistic Predictions . . . . . 7

**5. SVM** **9**

5.1 Tuning . . . . . 9

5.2 SVM Predictions . . . . . 11

**6. KNN** **13**

6.1 Cross-Validation . . . . . 13

6.2 KNN Predictions . . . . . 14

**7. Trees** **15**

7.1 Pruning and Cross-Validation . . . . . 18

7.2 Leave-One-Out Cross-Validation . . . . . 19

7.3 Tree Predictions . . . . . 21

**8. Bagging** **24**

8.1 Bagging predictions . . . . . 24

**9. Random Forest** **26**

9.1 Random Forest predictions . . . . . 29

**10. Models comparison** **30**

```
# Libraries
library(dplyr)
library(corrplot)
library(e1071)
library(caret)
library(ggplot2)
library(pROC)
library(ROCR)
library(tree)
library(randomForest)
library(gbm)
library(knitr)
library(tidyr)
```

# 1. Descriptive Analysis

## 1.1 The dataset

The dataset (link) is a collection of medical and demographic data from patients, along with their diabetes status (positive or negative). Understanding and analyzing diabetes occurrences can be useful for healthcare professionals in identifying patients who may be at risk of developing diabetes and in developing personalized treatment plans. Additionally, the dataset can be used by researchers to explore the relationships between various medical and demographic factors and the likelihood of developing diabetes.

The dataset consists of 100000 observations and 9 variables:

- `diabetes` : binary variable, with values of 1 indicating the presence of diabetes in the individual and 0 indicating the absence
- `gender` : numerical variable referring to the biological sex of the individual, which can have an impact on their susceptibility to diabetes
- `age` : numerical variable referring to the age of the individual. It ranges between $[0, 80]$. Diabetes is more commonly diagnosed in older adults
- `hypertension` : binary variable, with values of 1 indicating the presence of hypertension in the individual and 0 the absence. Hypertension is a medical condition in which the blood pressure in the arteries is persistently elevated
- `heart_disease` : binary variable, with values of 1 indicating the presence of a heart disease in the individual and 0 the absence. Heart disease is another medical condition that is associated with an increased risk of developing diabetes
- `smoking_history` : categorical variable, with 5 categories: never, not current, former, current and ever. Smoking history is also considered a risk factor for diabetes and it can exacerbate the complicaions associated with diabetes
- `bmi` : numerical variable referring to the Body Mass Index of the individual. Higher BMI are linked to a higher risk of diabetes. It ranges between $[10.16, 71.55]$

$$\begin{cases} underweight & \text{if } bmi < 18.5 \\ normal & \text{if } 18.5 \leq bmi < 25 \\ overweight & \text{if } 25 \leq bmi < 30 \\ obese & \text{if } bmi \geq 30 \end{cases}$$

- `HbA1c_level` : numerical variable referring to the Hemoglobin A1c level of the individual. It is a measure of the average blood sugar level over the last 2-3 months. Mainly, an HbA1c level above 6.5% indicates diabetes.
- `blood_glucose_level` : numerical variable referring to the amount of glucose in the bloodstream of the individual at a given time. High blood glucose levels are a key indicator of diabetes.

### 1.2 Goal of the project

Diabetes is a chronic metabolic disorder characterized by elevated blood sugar levels, which can lead to serious health complications if not managed effectively. Early detection and classification of diabetic patients are crucial for timely intervention and personalized treatment strategies. In this project, our goal is to develop a classification model to distinguish between diabetic and non-diabetic patients using various statistical methods. By analyzing a comprehensive dataset containing clinical and demographic information of patients, we aim to identify key predictors and patterns associated with diabetes. Through rigorous statistical analysis and model validation, the goal is to build a reliable predictive model capable of accurately classifying patients into diabetic and non-diabetic categories. This research has the potential to enhance diagnostic accuracy, improve patient outcomes, and contribute to the development of targeted interventions for individuals at risk of diabetes.

## 2. Data preparation

### 2.1 Check for NA

The dataset underwent an examination to identify and address any missing values, with the aim of ensuring data completeness. While no missing values were detected, instances of *No Info* entries were found within the `smoking_history` variable. Consequently, these entries were removed from the dataset to uphold data quality standards. As a result of this filtering process, the dataset was reduced to 64184 observations.

```
print(nrow(data))
```

```
## [1] 100000
```

```
data=na.omit(data)
print(nrow(data))
```

```
## [1] 100000
```

```
data = filter(data, smoking_history != "No Info")
print(nrow(data))
```

```
## [1] 64184
```

### 2.2 Dataset reduction

Following the filtering process, a subset of 5000 observations was randomly selected to enhance computational efficiency. Therefore, the analysis will focus on this subset of 5000 observations.

```
index <- sample(nrow(data), 5000, replace = FALSE)
data <- data[index, ]
```

### 2.3 Categorical variables

In this section, it will be addressed the treatment of categorical variables within the dataset. Many machine learning algorithms, including SVM and logistic regression, require numerical input data. Consequently, categorical variables must be appropriately transformed into numerical representations.

To accomplish this, the chosen method involves mapping each category within a categorical variable to a unique numerical value. This process is applied to the `gender` and `smoking_history` variables as follows.

For the `gender` variable, the categories *Male* and *Female* are converted to binary values, with *Male* represented as 1 and *Female* as 0.

```
data$gender = ifelse(data$gender == "Male", 1, 0)
```

Similarly, each category of the `smoking_history` variable (*never*, *not current*, *former*, *current* and *ever*) is mapped to a numerical value ranging from 0 to 4, enabling the representation of different smoking statuses in a quantitative format.

```
data$smoking_history <- ifelse(data$smoking_history == "never", 0,
                               ifelse(data$smoking_history == "not current", 1,
                                      ifelse(data$smoking_history == "former", 2,
                                             ifelse(data$smoking_history == "current", 3, 4))))
```

## 3. Plots

The distribution of diabetes cases within the subset of the dataset is visually depicted. The primary objective is to illustrate the balance between individuals diagnosed with diabetes and those without the condition.

```
count <- table(data$diabetes)

barplot(count, xlab = "Diabetes", ylab = "Count", ylim=c(0, 5900),
        col=c("lightsteelblue3", "rosybrown3"))

text(x = c(1, 2), y = count, labels = count, pos = 3, cex = 0.8, col = "black")
```

From the previous plot it is possible to visualize the updated frequency distribution of `diabetes` which indicates that within the subset of individuals there are 517 cases of diabetes and 4483 cases of non-diabetes, highlighting a significant imbalance in the final dataset.

The correlation matrix provides a comprehensive overview of the relationships between variables in the dataset. From the following plot it can be seen that there is a positive linear relationship between the response variable `diabetes` and the predictors `blood_glucose_level` and `HbA1c_level`. Additionally, there seems to be a positive (but quite weak) linear relationship between `age` and all the other variables (except for, of course, `gender`).

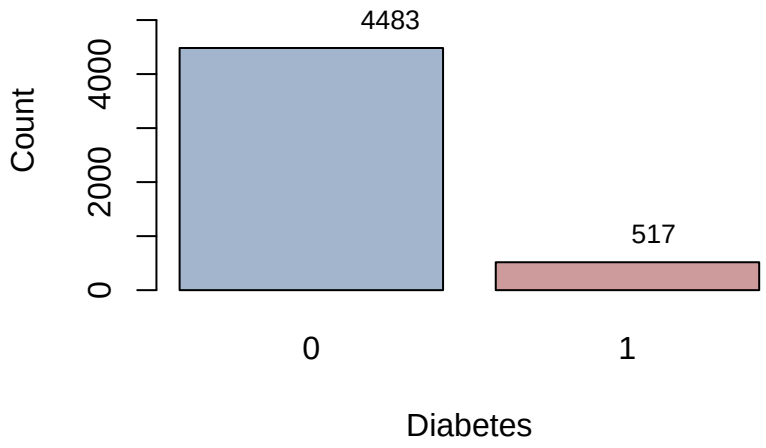



Figure 1: Count of diabetes occurences

```
corr_matrix = cor(data)
corrplot(corr_matrix, method="circle")
```

## 4. Logistic regression

Logistic regression, a commonly used statistical method for classification tasks, is particularly effective for analyzing binary outcomes, such as the presence or absence of diabetes in individuals. This approach employs a parametric model known as the *logit function* to estimate the probability of the occurrence of an event given certain input variables. In this context, the input variables include `gender`, `age`, `hypertension`, `heart_disease`, `smoking_history`, `bmi`, `HbA1c_level` and `blood_glucose_level`, denoted as $X$, while the binary outcome variable, denoted as $Y$, is `diabetes`. By directly modeling the estimated probability $P(Y|X)$, logistic regression provides insights into the relationship between these input variables and the binary outcome.

The dataset is split into training set and test set for model comparison.

```
# Creation of the train and test sets
data$diabetes=as.factor(data$diabetes)
n=nrow(data)
select.test = sample(1:n,n/10)

train = as.data.frame(data[-select.test,])
test = as.data.frame(data[select.test,])
```

Beginning with a logistic regression based on a linear model.

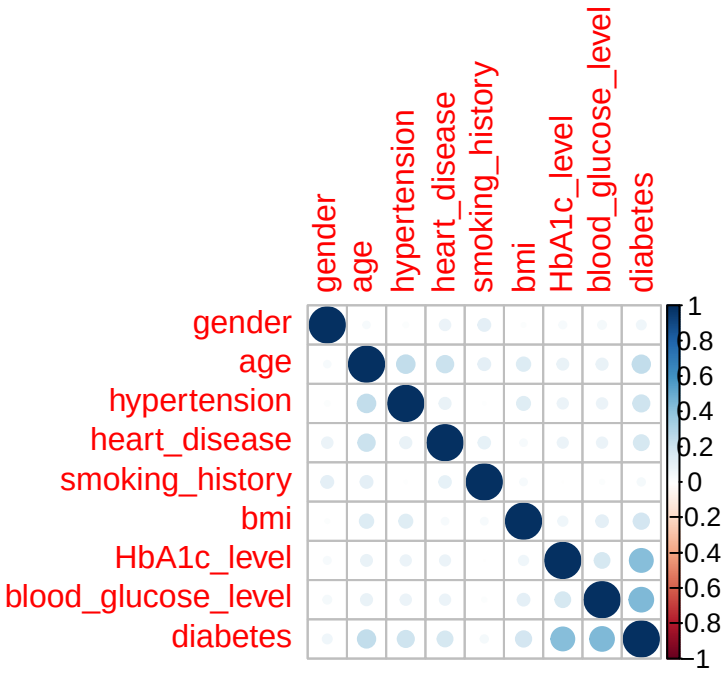



Figure 2: Correlation Matrix

```
logit = glm(diabetes~.,data=train,family='binomial')
summary(logit)
```

```
##
## Call:
## glm(formula = diabetes ~ ., family = "binomial", data = train)
##
## Coefficients:
##                       Estimate Std. Error z value Pr(>|z|)
## (Intercept)         -26.296969   1.205621 -21.812  < 2e-16 ***
## gender                0.221196   0.156707   1.412   0.1581
## age                   0.048523   0.005099   9.517  < 2e-16 ***
## hypertension          1.108529   0.190571   5.817 6.00e-09 ***
## heart_disease         0.672021   0.271399   2.476   0.0133 *
## smoking_history       0.047304   0.057933   0.817   0.4142
## bmi                   0.068649   0.010751   6.385 1.71e-10 ***
## HbA1c_level           2.177904   0.146413  14.875  < 2e-16 ***
## blood_glucose_level   0.035808   0.002170  16.501  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 3008.4  on 4499  degrees of freedom
## Residual deviance: 1207.3  on 4491  degrees of freedom
## AIC: 1225.3
##
## Number of Fisher Scoring iterations: 8
```

The logistic regression analysis reveals that several predictors significantly contribute to explaining the response variable. Specifically, the intercept , `age`, `HbA1c_level`, `bmi` and `blood_glucose_level` exhibit very small p-value. This result is coherent with established medical knowledge, where diabetes commonly occurs in older individuals with higher BMI and elevated blood glucose levels. Therefore, the findings are currently confirming what is known in the scientific literature.

**Coefficients Interpretation**

- The coefficient estimate for `age` is approximately 0.048523 with a standard error of 0.005099. This suggests that for each one-unit increase in age, the log odds of having diabetes increase by approximately 0.048523, holding all other variables constant which is perfectly logic.
- The coefficient estimate for `blood_glucose_level` is approximately 0.035808 with a standard error of 0.002170. This indicates that for each one-unit increase in blood glucose level, the log odds of having diabetes increase by approximately 0.035808, holding all other variables constant.

## 4.1 Logistic Predictions

It is now performed prediction according to the logit model.

```
yhat.logit = predict(logit,newdata=test,type='response')
prediction.logit = factor(ifelse(yhat.logit>0.5,1,0))
```

The confusion matrix provides a snapshot of the performance of the logit model.

```
cf_log = confusionMatrix(data=prediction.logit,test$diabetes)
cf_log
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   0   1
##          0 441  18
##          1  11  30
##
##                Accuracy : 0.942
##                  95% CI : (0.9178, 0.9608)
##     No Information Rate : 0.904
##     P-Value [Acc > NIR] : 0.001433
##
##                   Kappa : 0.6425
##
##  Mcnemar's Test P-Value : 0.265205
##
##             Sensitivity : 0.9757
##             Specificity : 0.6250
##          Pos Pred Value : 0.9608
##          Neg Pred Value : 0.7317
##              Prevalence : 0.9040
##          Detection Rate : 0.8820
##    Detection Prevalence : 0.9180
##       Balanced Accuracy : 0.8003
##
```

```
##         'Positive' Class : 0
##
```

The model achieves an accuracy of 94.2%, indicating proficiency in classifying the absence/presence of diabetes in individuals. The proportion of actual non-diabetes cases that were correctly identified is around 97.6%. Additionally, the proportion of actual diabetes cases that were correctly identified is around 62.5%. This suggests the model performs well in accurately classifying non-diabetes cases, but there could be potential for enhancement in detecting diabetes cases.

It is now created a heatmap in order to visualize and better understand the previous results.

```r
cflog_df <- as.data.frame(as.table(cf_log))
colnames(cflog_df) <- c("Predicted", "Reference", "Count")

# Sum for the actual labels
sums <- tapply(cflog_df$Count, cflog_df$Reference, sum)

# Percentages
cflog_df$Percentage <- cflog_df$Count / sums[cflog_df$Reference] * 100

ggplot(cflog_df, aes(x = Reference, y = Predicted, fill = Percentage)) +
  geom_tile(color = "white") +
  geom_text(aes(label = paste0(round(Percentage, 1), "%")), size = 3) +
  scale_fill_gradient(low = "white", high = "pink") +
  theme_minimal() +
  labs(x = "Actual", y = "Predicted", title = "Normalized Confusion Matrix")
```

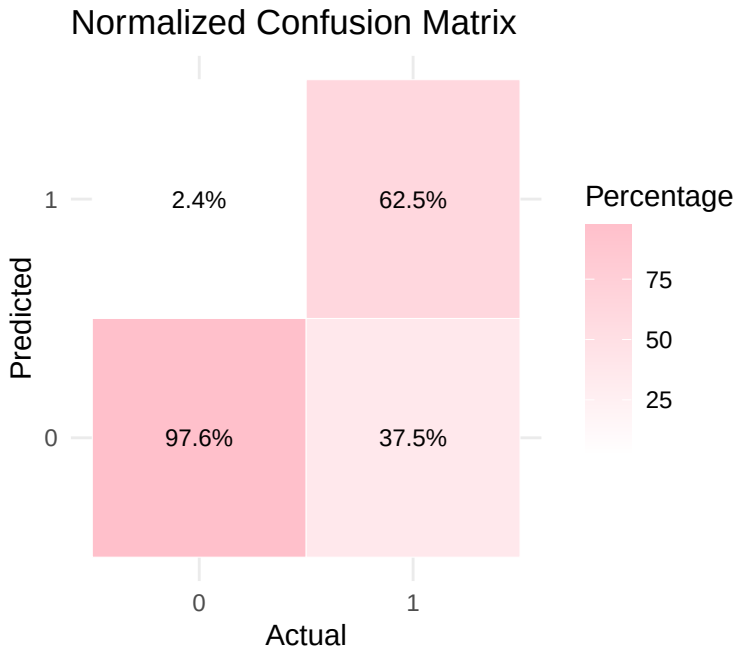



Figure 3: Heatmap of Predicted Values vs Actual Values - Logit Model

Overall, the model demonstrates better accuracy in identifying non-diabetes cases than diabetes cases. However, it is important to note that the model misclassified 11 individuals without diabetes as diabetic,

which could pose challenges in clinical contexts where false diagnoses carry significant consequences, even though it is a small percentage.

# 5. SVM

Support Vector Machines (SVM) is an approach for binary classification. SVM aims to find the optimal hyperplane that separates the two classes within the feature space while maximizing the margin. The margin refers to the distance between the hyperplane and the closest data points from each class, known as support vectors. By maximizing the margin, SVMs not only aim for accurate predictions on the training data but also for robust generalization to unseen data. In this case SVM accomodates non-linear decision boundaries through the use of the radial kernel function. Note that SVM exclusively operates with numerical variables. Consequently, the number of variables considered will be reduced compared to the original dataset. Additionally, it can be very sensible to different scale of measurements.

```
# Creation of the train and test sets
n=nrow(data)
select.test = sample(1:n,n/10)

train_quant = as.data.frame(data.scaled [-select.test,])
test_quant = as.data.frame(data.scaled [select.test,])
```

## 5.1 Tuning

It is now performed hyperparameter tuning for SVM using radial kernel to predict diabetes based on various features. The tuning process employed 10-fold cross-validation and it is essential for optimizing the performance of SVM by selecting the best combination of parameters *cost* and *gamma*. *cost* and *gamma* are used to control the trade-off between model complexity and the ability to fit the training data.

- *cost* represents the penalty parameter for misclassification errors in the SVM model. Higher the cost parameter, greater the penalty for misclassifying training examples, leading to a decision boundary that closely follows the training data. Consequently, the decision boundary will have a smaller margin, as it attemps to separate classes with high precision
- *gamma* is the parameter that influences the flexibility of the SVM model. A higher value of *gamma* indicates a narrower influence radius for each training example, meaning that the SVM model will be more influenced only by data points close to the separation hyperplane.

```
tune.out=tune(svm,diabetes~.,data=train_quant,kernel="radial",
              ranges=list(cost=c(0.001, 0.01, 0.1, 1,5,10,100),gamma=c(0.5,1,2,3,4) ))

summary(tune.out)
```

```
##
## Parameter tuning of 'svm':
##
## - sampling method: 10-fold cross validation
##
## - best parameters:
##  cost gamma
##    10     1
##
```

```
## - best performance: 0.04022222
##
## - Detailed performance results:
##     cost gamma      error dispersion
## 1  1e-03   0.5 0.10444444 0.01525279
## 2  1e-02   0.5 0.10444444 0.01525279
## 3  1e-01   0.5 0.05155556 0.01122412
## 4  1e+00   0.5 0.04311111 0.01158022
## 5  5e+00   0.5 0.04266667 0.01174956
## 6  1e+01   0.5 0.04133333 0.01043367
## 7  1e+02   0.5 0.04066667 0.01118739
## 8  1e-03   1.0 0.10444444 0.01525279
## 9  1e-02   1.0 0.10444444 0.01525279
## 10 1e-01   1.0 0.06577778 0.01437113
## 11 1e+00   1.0 0.04377778 0.01176123
## 12 5e+00   1.0 0.04044444 0.01087654
## 13 1e+01   1.0 0.04022222 0.01090928
## 14 1e+02   1.0 0.04488889 0.01092687
## 15 1e-03   2.0 0.10444444 0.01525279
## 16 1e-02   2.0 0.10444444 0.01525279
## 17 1e-01   2.0 0.09800000 0.01526896
## 18 1e+00   2.0 0.04644444 0.01223680
## 19 5e+00   2.0 0.04688889 0.01144918
## 20 1e+01   2.0 0.04733333 0.01098946
## 21 1e+02   2.0 0.05822222 0.01369068
## 22 1e-03   3.0 0.10444444 0.01525279
## 23 1e-02   3.0 0.10444444 0.01525279
## 24 1e-01   3.0 0.10444444 0.01525279
## 25 1e+00   3.0 0.05200000 0.01253582
## 26 5e+00   3.0 0.05222222 0.01231725
## 27 1e+01   3.0 0.05400000 0.01300206
## 28 1e+02   3.0 0.07088889 0.01824840
## 29 1e-03   4.0 0.10444444 0.01525279
## 30 1e-02   4.0 0.10444444 0.01525279
## 31 1e-01   4.0 0.10444444 0.01525279
## 32 1e+00   4.0 0.06311111 0.01382628
## 33 5e+00   4.0 0.06022222 0.01318643
## 34 1e+01   4.0 0.06311111 0.01507912
## 35 1e+02   4.0 0.08266667 0.01556613
```

The best parameters found were a cost value of 10 and a gamma value of 1. These parameters correspond to the point of minimum error during cross-validation.

Therefore, the best-tuned SVM model for diabetes prediction is the one trained using a radial kernel with cost set to 10 and gamma set to 0.5.

```
bestmod.SVM=tune.out$best.model
summary(bestmod.SVM)
```

```
##
## Call:
## best.tune(METHOD = svm, train.x = diabetes ~ ., data = train_quant,
##     ranges = list(cost = c(0.001, 0.01, 0.1, 1, 5, 10, 100), gamma = c(0.5,
##         1, 2, 3, 4)), kernel = "radial")
```

```
##
##
## Parameters:
##    SVM-Type:  C-classification
##  SVM-Kernel:  radial
##        cost:  10
##
## Number of Support Vectors:  723
##
##  ( 452 271 )
##
##
## Number of Classes:  2
##
## Levels:
##  0 1
```

## 5.2 SVM Predictions

It is now performed prediction according to the best tuned svm model.

```
prediction.SVM = predict(bestmod.SVM,newdata=test_quant)
```

The confusion matrix provides a snapshot of the performance of the SVM model.

```
confusion_matrix_SVM = confusionMatrix(data=prediction.SVM,test_quant$diabetes)
print(confusion_matrix_SVM)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   0   1
##          0 450  14
##          1   3  33
##
##                Accuracy : 0.966
##                  95% CI : (0.9461, 0.9801)
##     No Information Rate : 0.906
##     P-Value [Acc > NIR] : 1.664e-07
##
##                   Kappa : 0.777
##
##  Mcnemar's Test P-Value : 0.01529
##
##             Sensitivity : 0.9934
##             Specificity : 0.7021
##          Pos Pred Value : 0.9698
##          Neg Pred Value : 0.9167
##              Prevalence : 0.9060
##          Detection Rate : 0.9000
##    Detection Prevalence : 0.9280
##       Balanced Accuracy : 0.8478
```

```
##
##        'Positive' Class : 0
##
```

The model achieves an accuracy of 96.6%, indicating proficiency in classifying the absence/presence of diabetes in individuals. The proportion of actual non-diabetes cases that were correctly identified is 99.3%. However, the proportion of actual diabetes cases that were correctly identified is lower, around 70.2%.

It is now created a heatmap in order to visualize and better understand the previous results.

```r
cfSVM_df <- as.data.frame(as.table(confusion_matrix_SVM))
colnames(cfSVM_df) <- c("Predicted", "Reference", "Count")

# Sum for the actual labels
sums <- tapply(cfSVM_df$Count, cfSVM_df$Reference, sum)

# Percentages
cfSVM_df$Percentage <- cfSVM_df$Count / sums[cfSVM_df$Reference] * 100

ggplot(cfSVM_df, aes(x = Reference, y = Predicted, fill = Percentage)) +
  geom_tile(color = "white") +
  geom_text(aes(label = paste0(round(Percentage, 1), "%")), size = 3) +
  scale_fill_gradient(low = "white", high = "steelblue") +
  theme_minimal() +
  labs(x = "Actual", y = "Predicted", title = "Normalized Confusion Matrix")
```

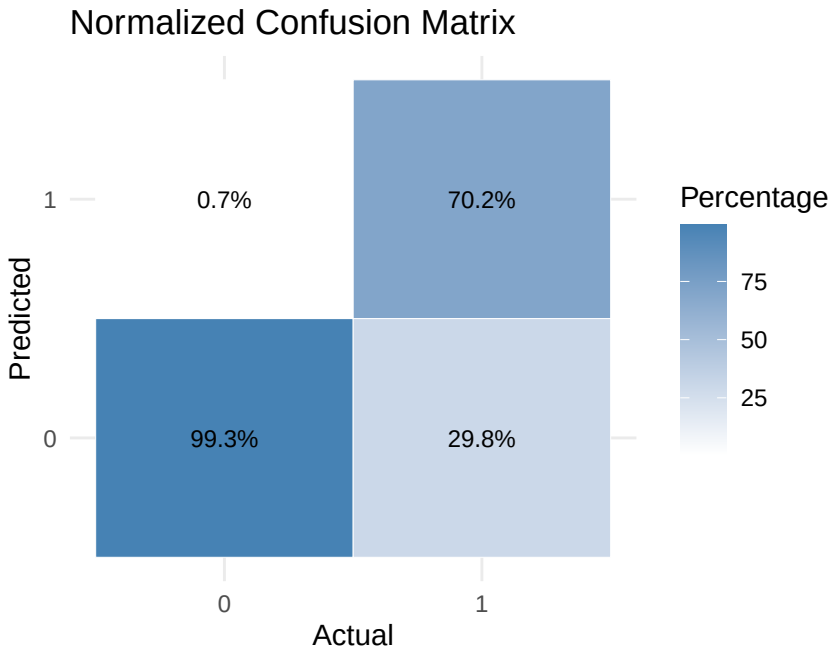



Figure 4: Heatmap of Predicted Values vs Actual Values - SVM

Overall, the model exhibits higher accuracy in correctly predicting non-diabetes cases compared to diabetes cases. Indeed, 70.2% of the diabetes cases were correctly predicted, considering the fact that the dataset is unbalanced, it is a quite high result. It is important to note that the model misclassified 3 individuals

without diabetes as diabetic, which could pose challenges in clinical contexts where false diagnoses carry significant consequences, even though it is a small percentage.

# 6. KNN

The K-Nearest Neighbors (KNN) is an approach for classification and regression. It is a non-parametric method that makes predictions based on the data included in neighborhoods of each point. The idea behind is that similar data points tend to have similar labels or outcomes. It relies on the fact that there exits a distance between covariates (Euclidean is often used). It is a classification method based on estimating the distribution of $Y|X$ and using the estimated probability to classify data.

$$\hat{P}[Y = c_j|X = x_i] = \frac{1}{K} \sum_{i \in N_i^K} I(y_i = c_j)$$

where:

- $\hat{P}[Y = c_j|X = x_i]$ is the posterior probability of class $j$
- $N_i$ is the neighborhood of $x_i$: $K$ points closest to $x_i$ (with respect to a distance $d$)

Note that KNN exclusively operates with numerical variables. Consequently, the number of variables considered will be reduced compared to the original dataset. Additionally, it can be very sensible to different scale of measurements. This preprocessing step aligns with the approach used for SVM. Consequently, the dataset used for KNN will remain unchanged.

## 6.1 Cross-Validation

The initial steps involve conducting a 10-fold cross-validation technique to identify the index (number of neighbors $k$) associated with the lowest cross-validation error.

```
# The number of neighbors to consider is 30
k.explore = 1:30

# Cross-validation on training set for choosing k
nfolds = 10 # Number of folds for cross-validation
folds = sample(1:nfolds,dim(train_quant)[1],replace = TRUE)
CVerror = matrix(ncol=length(k.explore),nrow=nfolds)

# 10-fold cross-validation to choose the optimal value of k
for(j in 1:nfolds){
  for(k in k.explore){
    prediction.KNN = class::knn(train_quant[which(folds!=j),1:4],train_quant[which(folds==j),1:4],cl=tra
    CVerror[j,k] = mean(prediction.KNN != train_quant[which(folds==j),5])
  }
}


plot(colMeans(CVerror),type='b',lwd=3, ylab = "Cross Validation Errors")
abline(v = which.min(colMeans(CVerror)), col = "red", lwd = 2)
```

The result of the plot indicates that the optimal number of neighbors selected through cross-validation is 11. Multiple iterations were conducted to ensure the stability of the outcome.

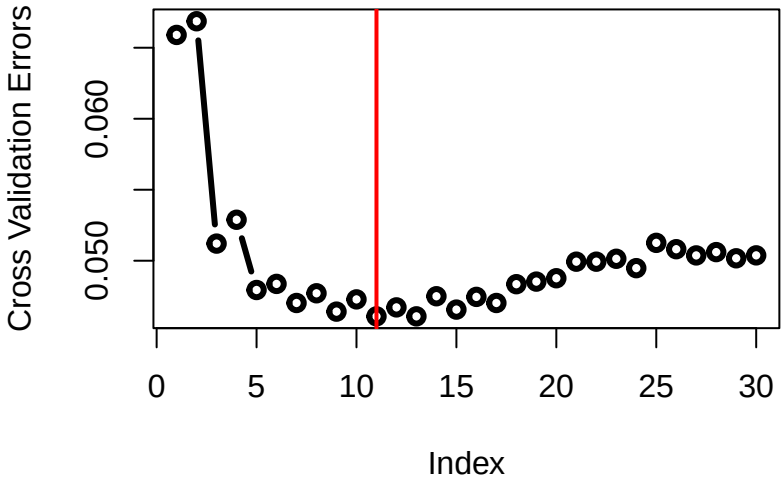



Figure 5: Cross Validation Error vs. Number of Neighbors

### 6.2 KNN Predictions

The final step involves using the KNN method to predict values and then comparing them to the true values using a confusion matrix.

```
# k = 11

min_index=which.min(colMeans(CVerror))

prediction.KNN = class::knn(train_quant[,1:4],test_quant[,1:4],cl=train_quant[,5],k=min_index)
cf_KNN=confusionMatrix(data=prediction.KNN,test_quant$diabetes)
print(cf_KNN)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   0   1
##          0 450  17
##          1   3  30
##
##                Accuracy : 0.96
##                  95% CI : (0.9389, 0.9754)
##     No Information Rate : 0.906
##     P-Value [Acc > NIR] : 3.338e-06
##
##                   Kappa : 0.729
##
##  Mcnemar's Test P-Value : 0.00365
##
```

```
##             Sensitivity : 0.9934
##             Specificity : 0.6383
##          Pos Pred Value : 0.9636
##          Neg Pred Value : 0.9091
##              Prevalence : 0.9060
##          Detection Rate : 0.9000
##    Detection Prevalence : 0.9340
##       Balanced Accuracy : 0.8158
##
##        'Positive' Class : 0
##
```

The model achieves an accuracy of 96%, indicating proficiency in classifying the absence/presence of diabetes in individuals. The proportion of actual non-diabetes cases that were correctly identified is almost 99.3%. However, the proportion of actual diabetes cases that were correctly identified is lower, 30 correctly classified cases out of 47 total cases of diabetes. This suggests that while the model performs well in identifying non-diabetes cases, it may benefit from further optimization to better capture diabetes cases.

It is now created a heatmap in order to visualize and better understand the previous results.

```
cfKNN_df <- as.data.frame(as.table(cf_KNN))
colnames(cfKNN_df) <- c("Predicted", "Reference", "Count")

# Sum for the actual labels
sums <- tapply(cfKNN_df$Count, cfKNN_df$Reference, sum)

# Percentages
cfKNN_df$Percentage <- cfKNN_df$Count / sums[cfKNN_df$Reference] * 100

ggplot(cfKNN_df, aes(x = Reference, y = Predicted, fill = Percentage)) +
  geom_tile(color = "white") +
  geom_text(aes(label = paste0(round(Percentage, 1), "%")), size = 3) +
  scale_fill_gradient(low = "white", high = "forestgreen") +
  theme_minimal() +
  labs(x = "Actual", y = "Predicted", title = "Normalized Confusion Matrix")
```

Overall, the model exhibits higher accuracy in correctly predicting non-diabetes cases compared to diabetes cases. This observation suggests that the model might struggle with correctly identifying individuals with diabetes, resulting in a higher rate of false negatives. It is important to note that the model misclassified 3 individuals without diabetes as diabetic, which could pose challenges in clinical contexts where false diagnoses carry significant consequences, even though it is a small percentage.

# 7. Trees

Given the binary nature of the response variable, a classification tree was selected to explore the relationships between covariates and the presence or absence of diabetes. This approach is commonly employed when the association between covariates and the response exhibits non-linear patterns.

```
tree.diabetes <- tree(train$diabetes ~. ,data= train ,method = 'gini')
summary(tree.diabetes)
```

```
##
```

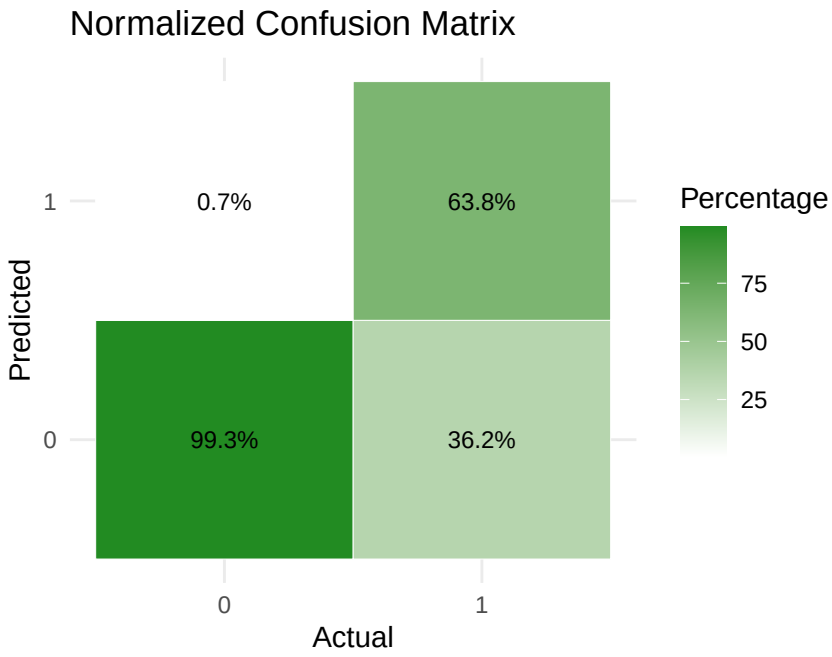



Figure 6: Heatmap of Predicted Values vs Actual Values - KNN

```
## Classification tree:
## tree(formula = train$diabetes ~ ., data = train, method = "gini")
## Variables actually used in tree construction:
## [1] "HbA1c_level"         "blood_glucose_level" "age"
## Number of terminal nodes:  6
## Residual mean deviance:  0.2084 = 936.4 / 4494
## Misclassification error rate: 0.03356 = 151 / 4500
```

From the output it is evident that:

- the variables `HbA1c_level`, `blood_glucose_level` and `age` were identified as the most important factors in constructing the tree;
- the tree comprises a total of 6 terminal nodes;
- regarding model fit, the residual mean deviance is 0.2084, representing the mean of the squared residuals.
- the misclassification error reveals a high accuracy (0.97), indicating a favorable outcome.

In the example, the classification tree is computed using the Gini criterion to evaluate which variables and splitting points are best for effectively separating observations into homogeneous groups with respect to the response variable `diabetes`.

```
plot(tree.diabetes,col='darkgray',lwd=2)
text(tree.diabetes,pretty=0,col='blue',cex=0.6)
```

From the plot, it is evident that the variable `HbA1c_level` serves as the primary determinant for the initial split, suggesting its significant role in explaining the variability of the response compared to the other variables.

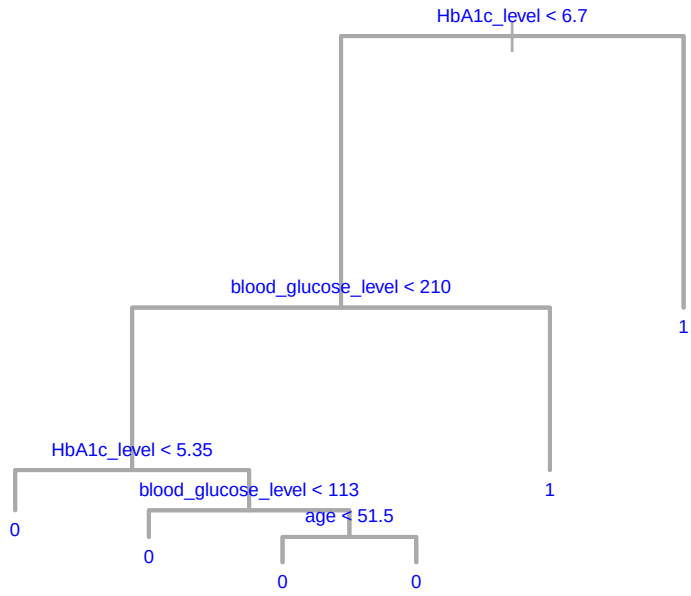



Figure 7: Classification Tree for Diabetes Prediction

Terminal nodes labeled as *1* indicate a majority of observations belonging to the diabetes class, while those labeled as *0* represent a majority in the non-diabetes class. These labels are determined by the mode of the output variable within each terminal node group.

The final classification is depicted by the leaves which display probabilities of having diabetes. For instance, individuals with an HbA1c level exceeding 6.7 exhibit a probability of 1 of having diabetes. Similarly, for those with an HbA1c level below 6.7 and blood glucose level surpassing 210, the probability of having diabetes remains 1.

```
# Logical scheme
tree.diabetes
```

```
## node), split, n, deviance, yval, (yprob)
##       * denotes terminal node
##
##  1) root 4500 3008.0 0 ( 0.89578 0.10422 )
##    2) HbA1c_level < 6.7 4287 1939.0 0 ( 0.94028 0.05972 )
##      4) blood_glucose_level < 210 4182 1300.0 0 ( 0.96389 0.03611 )
##        8) HbA1c_level < 5.35 1675    0.0 0 ( 1.00000 0.00000 ) *
##        9) HbA1c_level > 5.35 2507 1141.0 0 ( 0.93977 0.06023 )
##         18) blood_glucose_level < 113 714    0.0 0 ( 1.00000 0.00000 ) *
##         19) blood_glucose_level > 113 1793 1036.0 0 ( 0.91578 0.08422 )
##           38) age < 51.5 1025  264.0 0 ( 0.97171 0.02829 ) *
##           39) age > 51.5 768  672.4 0 ( 0.84115 0.15885 ) *
##      5) blood_glucose_level > 210 105    0.0 1 ( 0.00000 1.00000 ) *
##    3) HbA1c_level > 6.7 213    0.0 1 ( 0.00000 1.00000 ) *
```

## 7.1 Pruning and Cross-Validation

This section explores cross-validation techniques for pruning decision trees. Pruning serves to prevent overfitting by reducing tree complexity. Cross-validation assists in selecting the optimal tree size by evaluating its performance on various subsets of the data. To execute cross-validation, the dataset is partitioned into folds, with the default number of folds set to K=10.

```
# Cross-Validation for optimal tree pruning
cv.diabetes <- cv.tree(tree.diabetes, FUN=prune.misclass)

names(cv.diabetes)
```

```
## [1] "size"   "dev"    "k"      "method"
```

```
cv.diabetes
```

```
## $size
## [1] 6 3 2 1
##
## $dev
## [1] 151 151 256 469
##
## $k
## [1] -Inf    0  105  213
##
```

```
## $method
## [1] "misclass"
##
## attr(,"class")
## [1] "prune"         "tree.sequence"
```

The resulting cross-validation object, `cv.diabetes`, provides insights into the performance of the pruned tree across different levels.

- **size**: this vector denotes the sizes of the pruned trees, representing the number of terminal nodes. For example, the sequence "6,3,2,1" indicates that trees with 6,3,2 and 1 terminal node(s) were evaluated.
- **dev**: each element in this vector corresponds to the residual deviance of the pruned trees. Lower deviance values signify better model fit. For instance, the corresponding residual deviance for these trees are 151, 151, 256 and 469, indicating their goodness of fit to the data.
- **k**: this vector provides the number of model parameters (k) for each pruned tree. It helps in penalizing complex models during cross-validation. The sequence includes values for different terminal node configurations, with *-Inf* indicating the unpruned tree.
- **method**: it indicates the method used for cross-validation, which in this case is *misclass*.
- **attr(,"class")**: itspecifies the class of the returned objects, indicating that the output belongs to both *prune* and *tree.sequence* classes.

It is now made a comparison between cross-validation results using 10 folds and those using 100 folds to observe the impact of changing fold numbers on error rates.

```
par(mfrow = c(1, 2))
plot(cv.diabetes$size,cv.diabetes$dev,type="b", main = "C-V with 10 folds" ,xlab = "Tree size",ylab = "
min_error=min(cv.diabetes$dev)
min_tree_size = min(cv.diabetes$size[cv.diabetes$dev == min_error])
abline(v = min_tree_size, col = "red", lty = 2)

cv.diabetes_100 = cv.tree(tree.diabetes, K = 100, FUN = prune.misclass)
plot(cv.diabetes_100$size,cv.diabetes_100$dev,type="b", main = "C-V with 100 folds", xlab = "Tree size"
min_error_100 = min(cv.diabetes_100$dev)
min_tree_size_100 = min(cv.diabetes_100$size[cv.diabetes_100$dev == min_error_100])
abline(v = min_tree_size_100, col = "red", lty = 2)
```

The plots reveal that the optimal tree size, selected through cross-validation, remains consistent at 3, regardless of whether 10-fold or 100-fold cross-validation was employed. Multiple iterations were conducted to ensure the stability of the outcome.

## 7.2 Leave-One-Out Cross-Validation

In this section, it is explored the application of the Leave-One-Out Cross-Validation (LOOCV) in the context of decision tree pruning to validate the results obtained from previous C-V iterations. LOOCV is a robust technique and it involves training the model on $n - 1$ samples and evaluating it on the left-out sample, repeating this process for each observation.

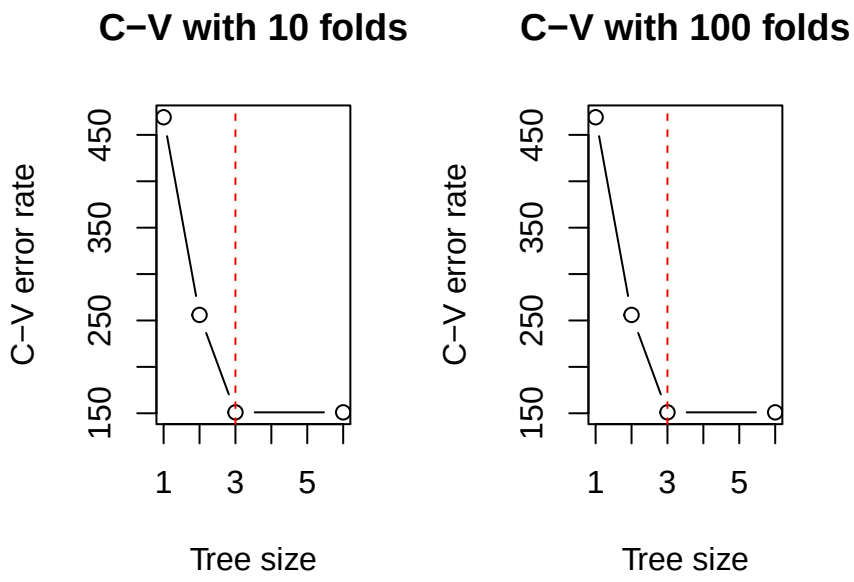



Figure 8: CV Comparison

```
cv.diabetes_LOOCV <- cv.tree(tree.diabetes, K = 5000, FUN =prune.misclass)

plot(cv.diabetes_LOOCV$size,cv.diabetes_LOOCV$dev,type="b", main = "LOOCV", xlab = "Tree size", ylab =
min_error_LOOCV=min(cv.diabetes_LOOCV$dev)
min_tree_size_LOOCV = min(cv.diabetes_LOOCV$size[cv.diabetes_LOOCV$dev == min_error_LOOCV])
abline(v = min_tree_size_100, col = "red", lty = 2)
```

The results of the previous C-V iterations and of the LOOCV coincide, showing that the optimal tree size is 3.

After determining the optimal tree size, the classification tree is pruned to achieve the best number of terminal nodes using the *misclass* method.

```
# Prune tree
prune.diabetes <- prune.tree(tree.diabetes,best=3, method = "misclass")
prune.diabetes
```

```
## node), split, n, deviance, yval, (yprob)
##       * denotes terminal node
##
## 1) root 4500 3008 0 ( 0.89578 0.10422 )
##   2) HbA1c_level < 6.7 4287 1939 0 ( 0.94028 0.05972 )
##     4) blood_glucose_level < 210 4182 1300 0 ( 0.96389 0.03611 ) *
##     5) blood_glucose_level > 210 105    0 1 ( 0.00000 1.00000 ) *
##   3) HbA1c_level > 6.7 213    0 1 ( 0.00000 1.00000 ) *
```

The displayed tree structure illustrates the hierarchical splits made by the classification prune tree algorithm to classify individuals as either diabetic (1) or non-diabetic (0). Each node represents a decision point based on a specific variable and threshold. Here is a breakdown of the interpretation:

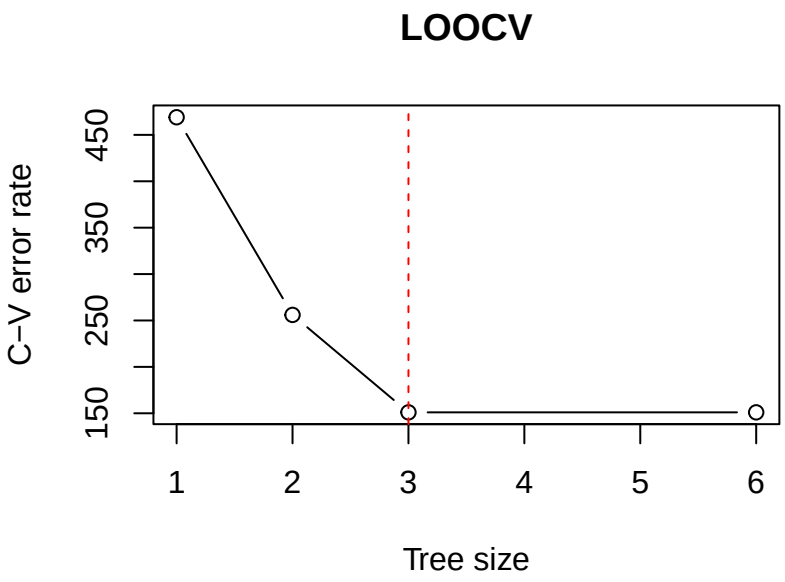



Figure 9: Leave-One-Out Cross-Validation

- **Root Node (Node 1)**: initially, the dataset is partitioned at the root node, where 4500 observations are evaluated. The predicted response indicates that 89.6% of cases belong to the non-diabetic class (0), while 10.4% belong to the diabetic class (1).
- **Node 2**: if the `HbA1c_level` variable is less than 6.7, the tree splits further into Node 2. In this node 4287 observations are evaluated. The predicted response shows that 94% of cases are non-diabetic (0), while 6% are diabetic (1).
- **Node 4**: for individuals with an `HbA1c_level` less than 6.7 and a `blood_glucose_level` less than 210, Node 4 is reached. Here, 4182 observations are examined The predicted response indicates that 96.4% of cases are non-diabetic (0), while 3.6% are diabetic (1). This node is a terminal node denoted by an asterisk (*), indicating no further splitting.
- **Node 5**: individuals with an `HbA1c_level` less than 6.7 and a blood glucose level greater than 210 are classified directly as diabetic (1) with a probability of 1. This is another terminal node, denoted by an asterisk (*).
- **Node 3**: if the `HbA1c_level` is greater than 6.7, all individuals are classified as diabetic (1) with a probability of 1. This, too, is a terminal node denoted by an asterisk (*).

It is now plotted the pruned tree to visualize the hierarchical splits made by the classification prune tree algorithm.

```
plot(prune.diabetes,col='darkgray',lwd=2)
text(prune.diabetes,pretty=0,col='blue',font=2)
```

### 7.3 Tree Predictions

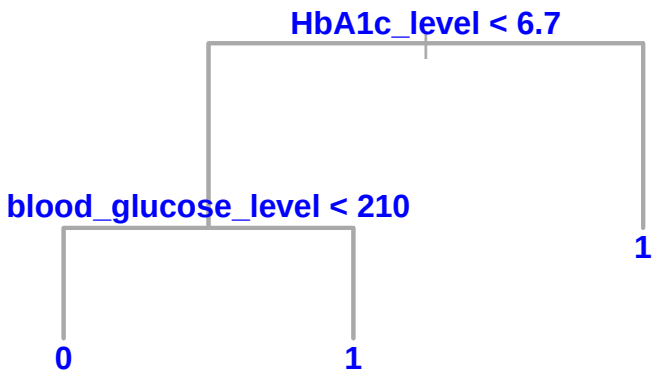



Figure 10: Pruned tree plot

```
tree_pred = predict(prune.diabetes, test, type = "class")
confusion_matrix_tree= confusionMatrix(data=tree_pred,test$diabetes)
print(confusion_matrix_tree)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   0   1
##          0 452  14
##          1   0  34
##
##                Accuracy : 0.972
##                  95% CI : (0.9535, 0.9846)
##     No Information Rate : 0.904
##     P-Value [Acc > NIR] : 2.229e-09
##
##                   Kappa : 0.8145
##
##  Mcnemar's Test P-Value : 0.000512
##
##             Sensitivity : 1.0000
##             Specificity : 0.7083
##          Pos Pred Value : 0.9700
##          Neg Pred Value : 1.0000
##              Prevalence : 0.9040
##          Detection Rate : 0.9040
##    Detection Prevalence : 0.9320
##       Balanced Accuracy : 0.8542
```

```
##
##        'Positive' Class : 0
##
```

The model achieves an accuracy of 97.2%, indicating proficiency in classifying the absence/presence of diabetes in individuals. The proportion of actual non-diabetes cases that were correctly identified is 100%, none of the non-diabetes cases were incorrectly identified as diabetes cases. Additionally, the proportion of actual diabetes cases that were correctly identified is around 70.8.

It is now created a heatmap in order to visualize and better understand the previous results.

```r
confusion_tree_df <- as.data.frame(as.table(confusion_matrix_tree))
colnames(confusion_tree_df) <- c("Predicted", "Reference", "Count")

# Sum for the actual labels
sums <- tapply(confusion_tree_df$Count, confusion_tree_df$Reference, sum)

# Percentages
confusion_tree_df$Percentage <- confusion_tree_df$Count / sums[confusion_tree_df$Reference] * 100

ggplot(confusion_tree_df, aes(x = Reference, y = Predicted, fill = Percentage)) +
  geom_tile(color = "white") +
  geom_text(aes(label = paste0(round(Percentage, 1), "%")), size = 3) +
  scale_fill_gradient(low = "white", high = "purple") +
  theme_minimal() +
  labs(x = "Actual", y = "Predicted", title = "Normalized Confusion Matrix")
```

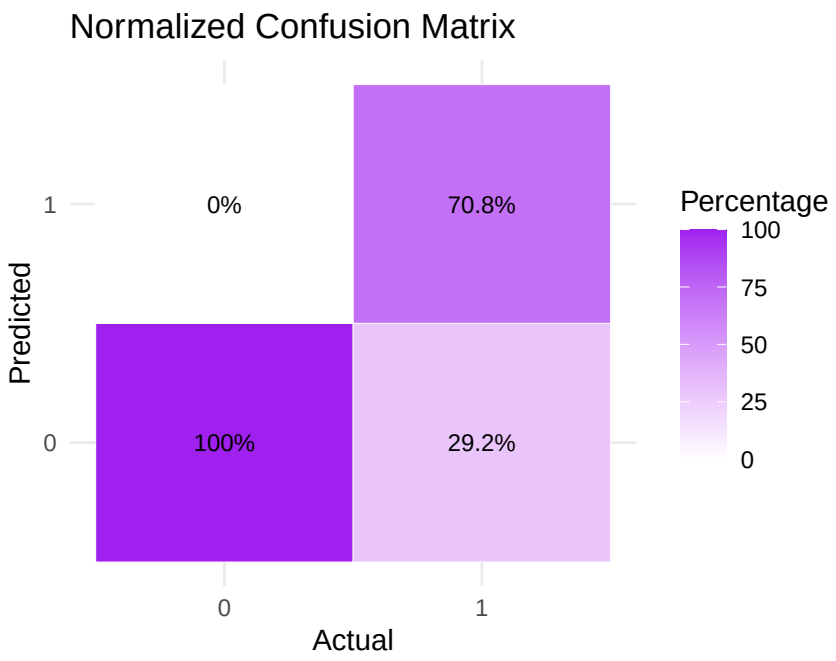



Figure 11: Heatmap of Predicted Values vs Actual Values - prune tree

Overall, the model demonstrates higher accuracy in correctly predicting non-diabetes cases compared to diabetes cases. Specifically, all non-diabetes cases were correctly classified. However, the proportion of correctly classified diabetes cases remains quite high at 70.8%, considering also that the dataset is unbalanced.

# 8. Bagging

It is now performed the Bagging (Boostrap Aggregating) technique using the *randomForest()* function considering all the covariates (`mtry = 8`) and employing a total of 1000 Bootstrap samples. This technique applies principles from the Bootstrap method and aims to produce more robust tree models by generating multiple bootstrapped datasets and training individual trees on each of them.

The Bagging process is made of three main steps:

- **Bootstrap Samples**: generate $B$ Bootstrap samples by randomly sampling $N$ observations with replacement from the original dataset.
- **Model Fitting**: fit a base model on each bootstrap sample: $\hat{f}^{*1}(\mathbf{x}), \hat{f}^{*2}(\mathbf{x}), \ldots, \hat{f}^{*B}(\mathbf{x})$.
- **Aggregation**: Combine the predictions from all base models using a suitable aggregation method. In the case of classification is performed the sum of the predictions of each model for each class, and the one with the highest sum is selected: $\hat{f}_{bag}(x) = \arg\max_j \sum_{b=1}^{B} I(f^{*b}(\mathbf{x}) = j)$.

```
bag_diabetes = randomForest(diabetes ~ ., data = train, mtry = 8, importance = TRUE,
                            ntree = 1000, type="classification")
bag_diabetes
```

```
##
## Call:
##  randomForest(formula = diabetes ~ ., data = train, mtry = 8,      importance = TRUE, ntree = 1000,
##                Type of random forest: classification
##                      Number of trees: 1000
## No. of variables tried at each split: 8
##
##         OOB estimate of  error rate: 3.64%
## Confusion matrix:
##      0   1 class.error
## 0 4007  24 0.005953858
## 1  140 329 0.298507463
```

From the obtained results, the OOB error rate reflects the average misclassification rate of observations that were excluded from the bootstrap sample used to train each individual tree. With an OOB error rate of 3.64%, the model demonstrates strong performance, indicating its ability to effectively capture underlying data patterns and make precise predictions on unseen instances.

## 8.1 Bagging predictions

Next, predictions obtained through the Bagging technique will be examined in order to allow for evaluating the model's performance and its predictive capability on unseen data.

```
predictions_bag = predict(bag_diabetes, newdata = test)
```

```
confusion_matrix_bag= confusionMatrix(data=predictions_bag, test$diabetes)
print(confusion_matrix_bag)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   0   1
##          0 448  11
##          1   4  37
##
##                Accuracy : 0.97
##                  95% CI : (0.951, 0.9831)
##     No Information Rate : 0.904
##     P-Value [Acc > NIR] : 7.874e-09
##
##                   Kappa : 0.8151
##
##  Mcnemar's Test P-Value : 0.1213
##
##             Sensitivity : 0.9912
##             Specificity : 0.7708
##          Pos Pred Value : 0.9760
##          Neg Pred Value : 0.9024
##              Prevalence : 0.9040
##          Detection Rate : 0.8960
##    Detection Prevalence : 0.9180
##       Balanced Accuracy : 0.8810
##
##        'Positive' Class : 0
##
```

The model achieves an accuracy of 97%, indicating proficiency in classifying the absence/presence of diabetes in individuals. The proportion of actual non-diabetes cases that were correctly identified is around 99%. It is important to note that the model misclassified 4 individuals without diabetes as diabetic, which could pose challenges in clinical contexts where false diagnoses carry significant consequences, even though it is a small percentage. Additionally, the proportion of actual diabetes cases that were correctly identified is 77%.

It is now created a heatmap in order to visualize and better understand the previous results.

```
cfbag_df <- as.data.frame(as.table(confusion_matrix_bag))
colnames(cfbag_df) <- c("Predicted", "Reference", "Count")

# Sum for the actual labels
sums <- tapply(cfbag_df$Count, cfbag_df$Reference, sum)

# Percentages
cfbag_df$Percentage <- cfbag_df$Count / sums[cfbag_df$Reference] * 100

ggplot(cfbag_df, aes(x = Reference, y = Predicted, fill = Percentage)) +
  geom_tile(color = "white") +
  geom_text(aes(label = paste0(round(Percentage, 1), "%")), size = 3) +
  scale_fill_gradient(low = "white", high = "darkorange") +
  theme_minimal() +
  labs(x = "Actual", y = "Predicted", title = "Normalized Confusion Matrix")
```

Overall, the model demonstrates higher accuracy in correctly predicting non-diabetes cases compared to diabetes cases. Specifically, just 4 of non-diabetes cases out of 452 (0.9%) were incorrectly classified as

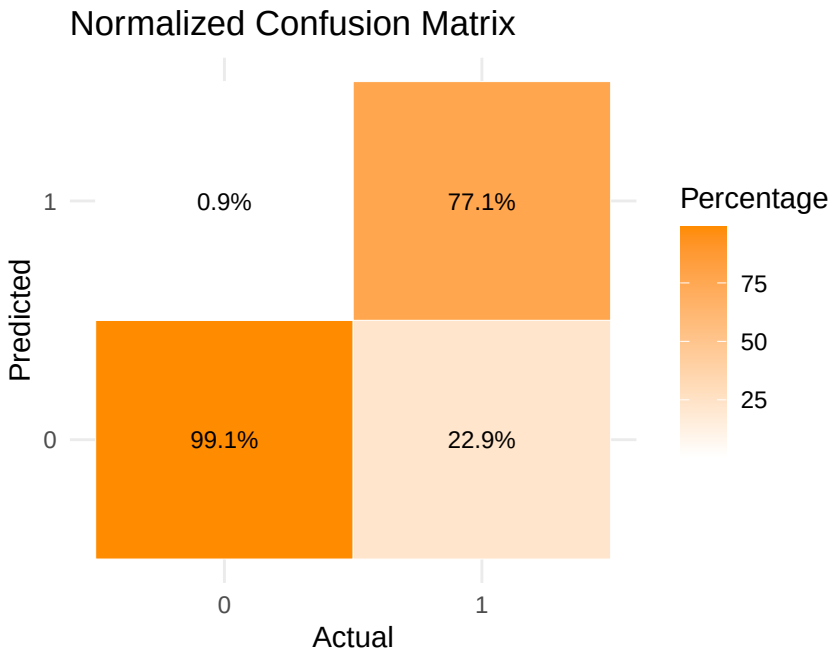



Figure 12: Heatmap of Predicted Values vs Actual Values - Bagging

diabetes-cases. However, the proportion of correctly classified diabetes cases remains high at 77.1%, considering also that the dataset is unbalanced.

## 9. Random Forest

It is now trained a Random Forest model with the aim of optimizing the selection of variables considered at each split of the classification trees. This technique allows to add variability to the trees generated in different runs, in order to decorrelate them. Furthermore, instead of evaluating different values of `mtry`, it will be selected the exact number of covariates to be considered at each split. Throughout this process, are utilized both Out-of-Bag (OOB) error estimation and Cross-Validation to determine the optimal number of covariates to select. Subsequently, the model will be fitted using the chosen `mtry` value to assess its performance.

The Random Forest process is made by the same steps of the Bagging process but unlike Bagging, where predictions are aggregated through majority voting, in Random Forest predictions are aggregated by averaging the outputs of all trees: $\hat{f}_{bag}(x) = \frac{1}{B}\sum_{b=1}^{B} f^{*b}(\mathbf{x})$.

This approach helps to reduce the variance of the model and provides a more stable prediction by leveraging the collective wisdom of multiple trees trained on different subsets of the data.

```
mtry.explore = 1:8
oob.err = double(length(mtry.explore))
nfolds = 10
folds = sample(1:nfolds, nrow(train), replace = TRUE)
cv.error = matrix(ncol = length(mtry.explore), nrow = nfolds)

for(mtry in 1:length(mtry.explore)) {
  rf = randomForest(diabetes ~ . , data = train, mtry = mtry.explore[mtry], ntree = 400, type = "classi
```

```
  oob.err[mtry] = rf$err.rate[nrow(rf$err.rate), "OOB"]

  for(j in 1:nfolds){
    rf = randomForest(diabetes ~ . , data = train[-which(folds == j), ], mtry = mtry.explore[mtry], ntr
    pred = predict(rf, train[which(folds == j), ], type = "response")
    cv.error[j, mtry] = mean(pred != train$diabetes[which(folds == j)])
  }

  cat(mtry, " ")
}
```

```
## 1  2  3  4  5  6  7  8
```

```
test.err = apply(cv.error, 2, mean)

importance(rf)
```

```
##                       MeanDecreaseGini
## gender                        9.960095
## age                          54.451253
## hypertension                  9.984522
## heart_disease                 6.611668
## smoking_history              20.262235
## bmi                          82.758033
## HbA1c_level                 358.815442
## blood_glucose_level         212.597104
```

```
varImpPlot(rf)
```

```
matplot(1:length(mtry.explore) , cbind(oob.err, test.err), pch = 19 , col = c("red", "blue"),
        type = "b", main = "OOB Error and CV Error", ylab = "Error Rate", xlab = "Number of Predictors
legend("topright", legend = c("Out of Bag Error", "CV Error"), pch = 19, col = c("red", "blue"))

min_cv_error <- min(oob.err)
min_mtry <- which.min(oob.err)

abline(v = min_mtry, col = "green", lwd = 2)
```

The results obtained from the *importance()* and *varImpPlot()* functions applied to the Random Forest model indicate that `HbA1c_level` and `blood_glucose_level` are the most influential variables in predicting the outcome. `HbA1c_level` exhibits the highest importance score of 358.815, followed closely by `blood_glucose_level` with a score of 212.597.

Observing the plot containing the Out-of-Bag (OOB) error and Cross-Validation (CV) error, a noticeable decline in error occurs at the level of the second variable, followed by a subsequent increase. In this case the observed pattern indicates that, as obtained from classification tree, the optimal choice for the number of covariates is two.

```
rf.2 = randomForest(diabetes ~ . , data = train, mtry = 2, ntree = 5000, type = "classification")
```




Figure 13: Variables Importance

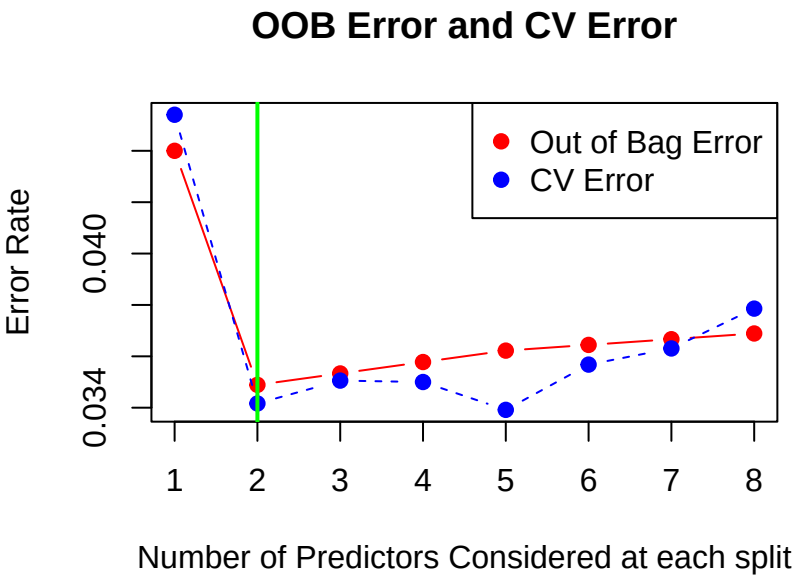



Figure 14: Variables Importance

## 9.1 Random Forest predictions

Now the analysis will delve into the predictions generated by the Random Forest technique to evaluate the model's performance and predictive accuracy on unseen data.

```
pred_rf = predict(rf.2, newdata = test)
confusion_matrix_rf = confusionMatrix(data=pred_rf, test$diabetes)
print(confusion_matrix_rf)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   0   1
##          0 450  14
##          1   2  34
##
##                Accuracy : 0.968
##                  95% CI : (0.9486, 0.9816)
##     No Information Rate : 0.904
##     P-Value [Acc > NIR] : 2.605e-08
##
##                   Kappa : 0.7924
##
##  Mcnemar's Test P-Value : 0.00596
##
##             Sensitivity : 0.9956
##             Specificity : 0.7083
##          Pos Pred Value : 0.9698
##          Neg Pred Value : 0.9444
##              Prevalence : 0.9040
##          Detection Rate : 0.9000
##    Detection Prevalence : 0.9280
##       Balanced Accuracy : 0.8520
##
##        'Positive' Class : 0
##
```

The model achieves an accuracy of 96.8%, indicating proficiency in classifying the absence/presence of diabetes in individuals. The proportion of actual non-diabetes cases that were correctly identified is 99.6%. It is important to note that the model misclassified 2 individuals without diabetes as diabetic, which could pose challenges in clinical contexts where false diagnoses carry significant consequences, even though it is a small percentage. Additionally, the proportion of actual diabetes cases that were correctly identified is 70.8%.

It is now created a heatmap in order to visualize and better understand the previous results.

```
cfrf_df <- as.data.frame(as.table(confusion_matrix_rf))
colnames(cfrf_df) <- c("Predicted", "Reference", "Count")

# Sum for the actual labels
sums <- tapply(cfrf_df$Count, cfrf_df$Reference, sum)

# Percentages
```

```r
cfrf_df$Percentage <- cfrf_df$Count / sums[cfrf_df$Reference] * 100

ggplot(cfrf_df, aes(x = Reference, y = Predicted, fill = Percentage)) +
  geom_tile(color = "white") +
  geom_text(aes(label = paste0(round(Percentage, 1), "%")), size = 3) +
  scale_fill_gradient(low = "white", high = "lightgreen") +
  theme_minimal() +
  labs(x = "Actual", y = "Predicted", title = "Normalized Confusion Matrix")
```

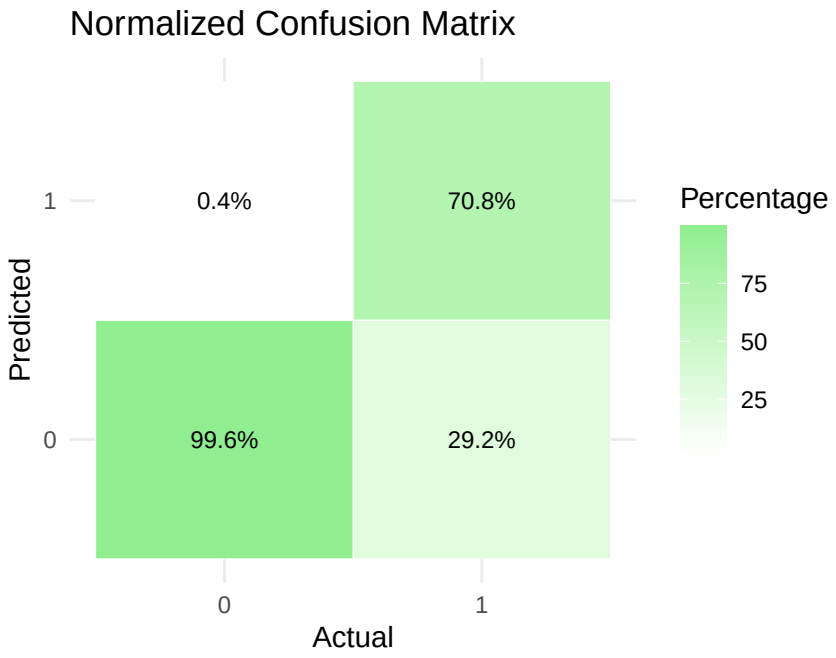



Figure 15: Heatmap of Predicted Values vs Actual Values - Random Forest

Overall, the model demonstrates higher accuracy in correctly predicting non-diabetes cases compared to diabetes cases. Specifically, all non-diabetes cases were correctly classified. However, the proportion of correctly classified diabetes cases remains high at 70.8%, considering also that the dataset is unbalanced.

## 10. Models comparison

Are now visualized the performance metrics of the six distinct models applied to the diabetes dataset. The evaluation focuses on accuracy, sensitivity, and specificity, providing insights into their predictive capabilities for identifying diabetic individuals.

```r
models <- c("Logit", "SVM", "KNN", "Trees", "Bagging", "Random Forest")
accuracy <- c(0.942, 0.966, 0.96, 0.972, 0.97, 0.968)
sensitivity <- c(0.9757, 0.9934, 0.9934, 0.9912, 1.00, 0.9956)
specificity <- c(0.6250, 0.7021, 0.6383, 0.7083, 0.7708, 0.7083)

results <- data.frame(Model = models, Accuracy = accuracy, Sensitivity = sensitivity, Specificity = spe

kable(results)
```

| Model | Accuracy | Sensitivity | Specificity |
|---|---|---|---|
| Logit | 0.942 | 0.9757 | 0.6250 |
| SVM | 0.966 | 0.9934 | 0.7021 |
| KNN | 0.960 | 0.9934 | 0.6383 |
| Trees | 0.972 | 0.9912 | 0.7083 |
| Bagging | 0.970 | 1.0000 | 0.7708 |
| Random Forest | 0.968 | 0.9956 | 0.7083 |

```
results_long <- pivot_longer(results, cols = c(Accuracy, Sensitivity, Specificity), names_to = "Metric"

ggplot(results_long, aes(x = Model, y = Value, fill = Metric)) +
  geom_bar(stat = "identity", position = "dodge") +
  geom_text(aes(label = round(Value, 2)), position = position_dodge(width = 0.95), vjust = -0.1) +
  labs(title = "Model Evaluation Metrics",
       x = "Model",
       y = "Value") +
  scale_fill_manual(values = c("Accuracy" = "#92c5de", "Sensitivity" = "#f4cae4", "Specificity" = "#b3e1
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

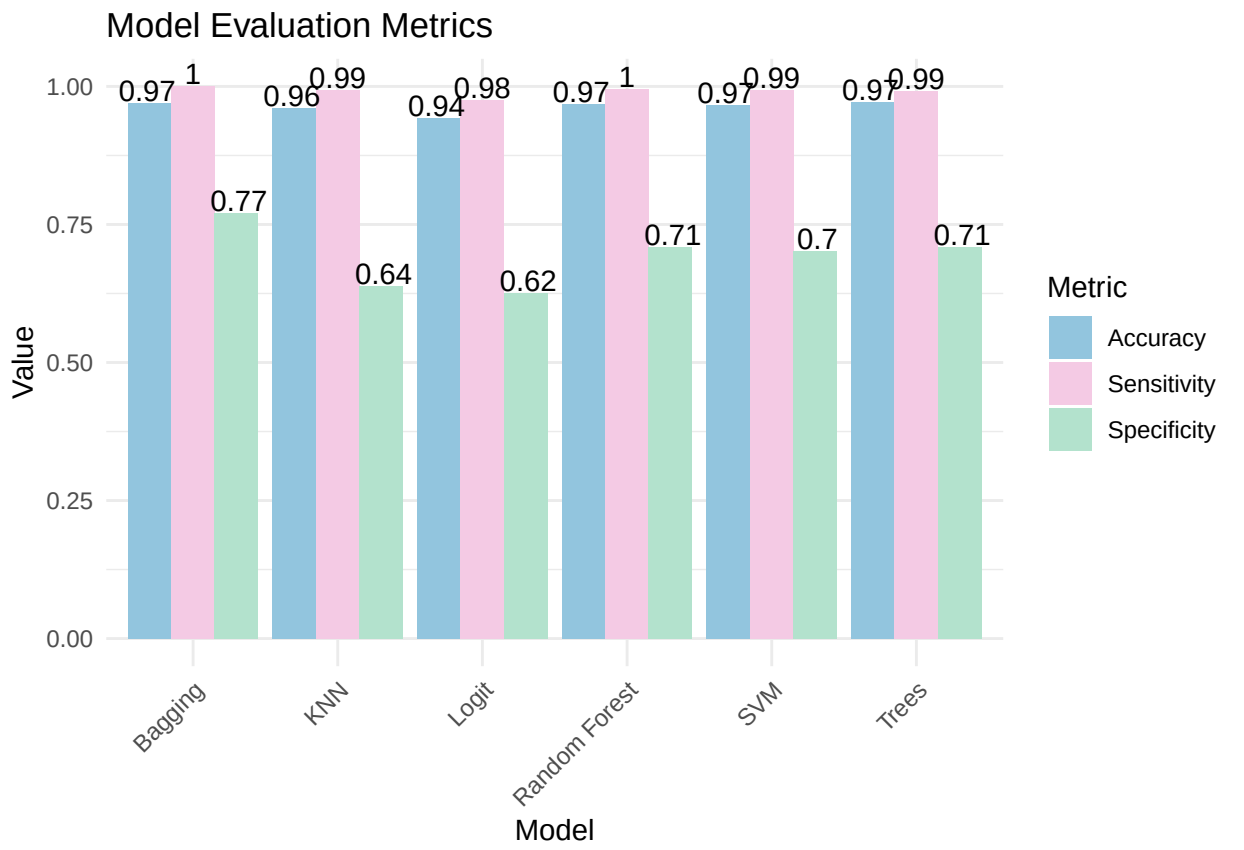



Figure 16: Final results table

The evaluation of six distinct models on the diabetes dataset provides valuable insights into their performance metrics. Across all models, consistent accuracy, sensitivity and specificity results were observed, indicating

robust performance in predicting diabetes.

Despite the satisfactory overall accuracy observed in all models, further analysis of the confusion matrix revealed certain limitations. Notably, in all models, there was a challenge in accurately predicting instances of diabetic individuals (class 1), as reflected in the sensitivity scores. It is worth considering that the challenge in accurately predicting instances of diabetic individuals (class 1) may be influenced by the fact that the dataset is unbalanced, resulting in a smaller number of observations for diabetic instances compared to non-diabetic ones (class 0). To address this issue, various models were developed with different configurations, aiming to improve the sensitivity while maintaining overall accuracy. However, despite the adjustments made in the models, including incorporating different features or utilizing ensemble techniques like Bagging and Random Forest, the sensitivity scores remained relatively consistent.

In overall examination of the performance metrics, the **Bagging**, the **Trees** and the **SVM** models emerged as the most promising options. The three models demonstrated commendable accuracy and specificity while maintaining competitive sensitivity scores. These findings suggest that further exploration and refinement of these models could lead to improved predictive capabilities for diabetic classification.